

编译原理实验报告

题目：对下面的文法对象，使用 c 语言构造它的预测分析程序；并任意给一算术表达式进行分析测试。

分析对象定义如下：

算术表达式 $\rightarrow$ 项 | 算术表达式 + 项 | 算术表达式 - 项

项 $\rightarrow$ 因式 | 项 * 因式 | 项 / 因式

因式 $\rightarrow$ 变量 | (算术表达式)

变量 $\rightarrow$ 字母

字母 $\rightarrow$ A|B|C|D|E|F|G|H|I|J|K|L|M|N|O|P|Q|R|S|T|U|V|W|X|Y|Z

一、分析

语法分析部分我们采用 LL(1) 方法实现，采用 LL(1) 方法实现语法分析要求文法满足以下要求：

一个文法能否用确定的自顶向下分析与文法中相同左部的每个产生式右部的开始符号集合有关，当有右部能 $\Rightarrow^* \epsilon$ 时则与其左部非终结符的后跟符号集合也有关，此外在产生式中不存在左递归即经过压缩，无左递归，无回溯。它的基本思想是从左到右扫描源程序，同时从识别符号开始生成句子的最左推导，并只向前查看一个输入符号，便能唯一确定应选择的规则。

下面将确切地定义满足确定的自顶向下分析条件的文法即 LL(1) 文法及 LL(1) 文法的判别并介绍如何对非 LL(1) 文法进行等价变换问题，也就是消除一个文法中的左递归和左公共因子。

注意：

一个文法中含有左递归和左公共因子绝对不是 LL(1) 文法，所以也就不可能用确定的自顶向下分析法，对此结论可以证明。然而，某些含有左递归和左公共因子的文法在通过等价变换把它们消除以后可能变为 LL(1) 文法，但需要用 LL(1) 文法的定义判别，也就是说文法中不含左递归和左公共因子，只是 LL(1) 文法的必要条件。

LL(1) 文法的定义（5 种定义）：

一个文法符号串的开始符号集合定义如下：

定义 1. 设 $G=(VT, VN, S, P)$ 是上下文无关文法， α 是任意的文法符号串， $FIRST(\alpha)$ 是从 α 推导出的串的开始符号的终结符集合。。。。

$FIRST(\alpha) = \{a | \alpha \Rightarrow^* a\beta, a \in VT, \alpha, \beta \in V^*\}$ 若 $\alpha \Rightarrow^* \epsilon$, 则规定 $\epsilon \in FIRST(\alpha)$ 。

当一个文法中相同左部非终结符的右部存在能 $\Rightarrow^* \epsilon$ 的情况则必须知道该非终结符的后跟符号的集合中是否含有其它右部开始符号集合的元素。为此，我们定义一个文法非终结符的后跟符号的集合如下：

定义 2. 设 $G=(VT, VN, S, P)$ 是上下文无关文法， $A \in VN$ ， S 是开始符号
 $FOLLOW(A) = \{a | S \Rightarrow^* \mu A \beta, \text{且 } a \in VT, a \in FIRST(\beta), \mu \in VT^*, \beta \in V^+\}$

若 $S \Rightarrow^* \mu A \beta$, 且 $\beta \in \epsilon$, 则 $\# \in \text{FOLLOW}(A)$ 。也可定义为: $\text{FOLLOW}(A) = \{a | S \Rightarrow^* \dots A a \dots, a \in VT\}$

若有 $S \Rightarrow^* \dots A$, 则规定 $\# \in \text{FOLLOW}(A)$

这里我们用 '#' 作为输入串的结束符, 或称为句子括号, 如: #输入串#。

定义 3. 给定上下文无关文法的产生式 $A \rightarrow \alpha$, $A \in VN, \alpha \in V^*$, 若 $\alpha \Rightarrow^* \epsilon$, 则 $\text{SELECT}(A \rightarrow \alpha) = \text{FIRST}(\alpha)$

如果 $\alpha \Rightarrow^* \epsilon$, 则 $\text{SELECT}(A \rightarrow \alpha) = \text{FIRST}(\alpha \epsilon) \cup \text{FOLLOW}(A)$ 。FIRST($\alpha \epsilon$) 表示 FIRST(α) 的非 $\{\epsilon\}$ 元素。

更进一步可以看出能够使用自顶向下分析技术必须使文法满足如下条件, 我们称满足条件的文法为 LL(1) 文法, 其定义为:

定义 4. 一个上下文无关文法是 LL(1) 文法的充分必要条件是:

对每个非终结符 A 的两个不同产生式, $A \rightarrow \alpha, A \rightarrow \beta$, 满足 $\text{SELECT}(A \rightarrow \alpha) \cap \text{SELECT}(A \rightarrow \beta) = \emptyset$, 其中 α, β 不同时能 ϵ 。

定义 5. LL (1) 文法也可定义为:

一个文法 G 是 LL (1) 的, 当且仅当对于 G 的每一个非终结符 A 的任何两个不同产生式 $A \rightarrow \alpha | \beta$, 下面的条件成立:

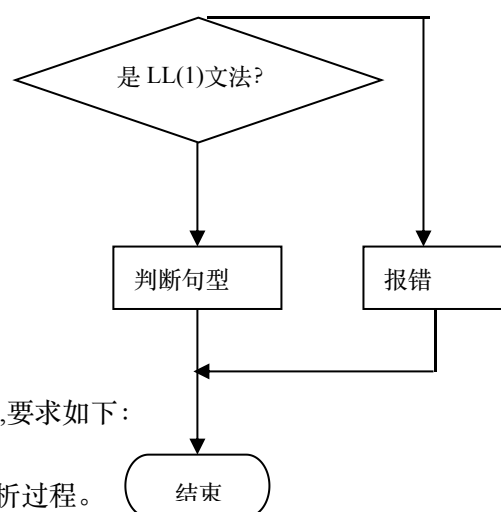
① $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$, 也就是 α 和 β 推导不出以某个相同的终结符 a 为首的符号串; 它们不应该都能推出空字 ϵ 。

② 假若 $\beta \Rightarrow^* \epsilon$ 那么, $\text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \emptyset$ 也就是, 若 $\beta \Rightarrow^* \epsilon$ 则 α 所能推出的串的首符号不应在 FOLLOW(A) 中。

二、算法

该程序可分为如下几步:

- (1) 读入文法
- (2) 判断正误
- (3) 若无误, 判断是否为 LL(1) 文法
- (4) 若是, 构造分析表;
- (5) 由总控算法判断输入符号串是否为该文法的句型。



根据下面 LL(1) 文法, 对输入串 w: (i+i)*(i+i)+i*i 进行 LL(1) 分析, 要求如下:

- 1、先手工建立 LL(1) 分析表;
- 2、分析输入串, 判断是否是语法上正确的句子, 并输出整个分析过程。

LL(1) 文法 G 为:

$E \rightarrow TE'$
 $E' \rightarrow +TE' | \epsilon$
 $T \rightarrow FT'$
 $T' \rightarrow *FT' | \epsilon$
 $F \rightarrow (E) | id$

分析算法:

输入: 串 w 和文法 G 的分析表 M。

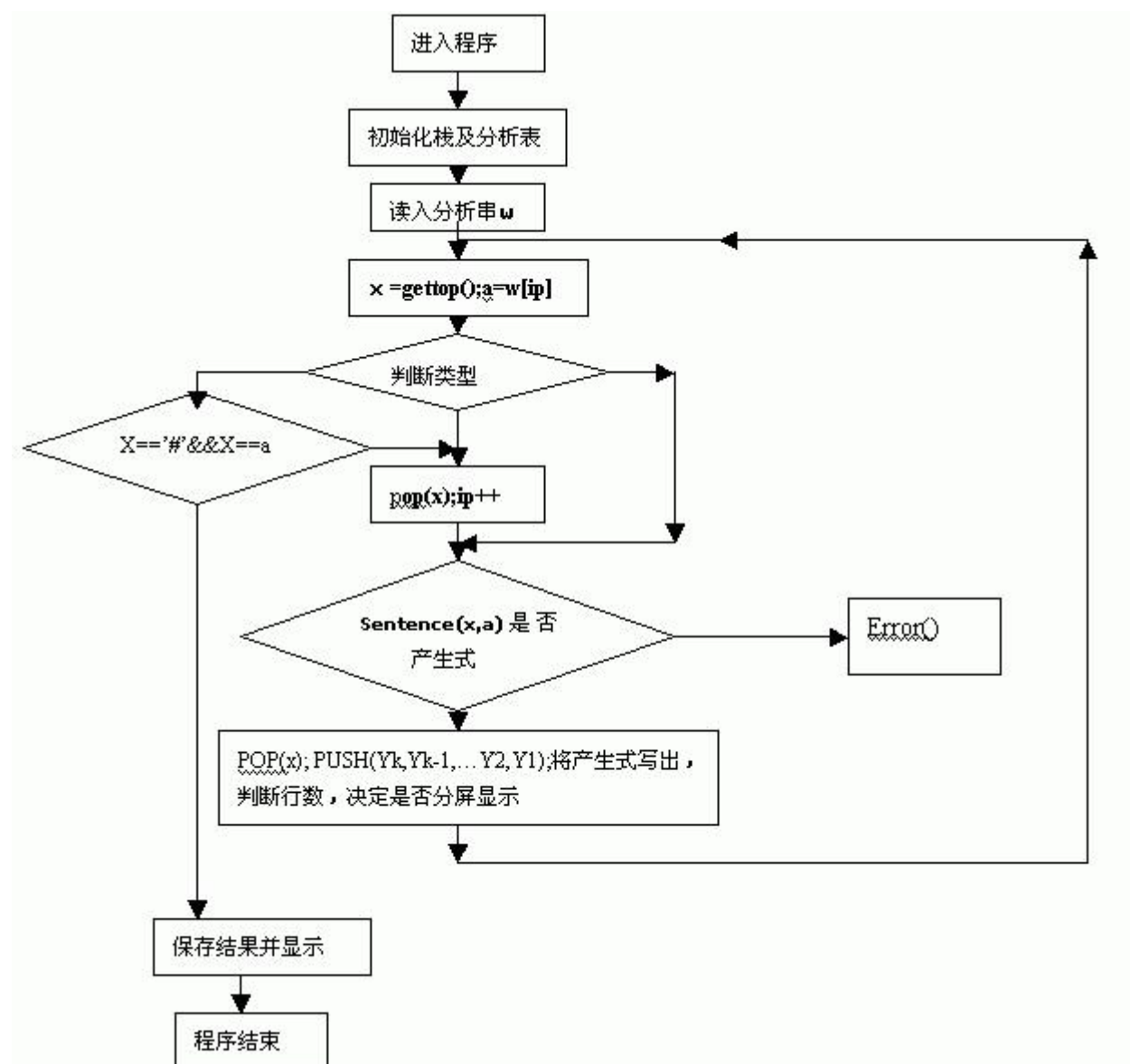
输出: 如果 W 属于 L (G), 则输出 W 的最左推导, 否则报告错误。

方法: 开始时, #S 在分析栈中, 其中 S 是文法的开始符号, 在栈顶; 令指针 ip 指向 W# 的

```

第一个符号; repeat
让 X 等于栈顶符号, a 为 ip 所指向的符号;
if X 是终结符号或# then
If X=a then 把 X 从栈顶弹出并使 ip 指向下一个输入符号
else error()
else /*X 是非终结符号*/
if M[x,a]=Xày1y2...yk then begin
从栈中弹出 X; 把 yk,yk-1,...,y1 压入栈, y1 在栈顶;
输出产生式 Xày1y2...yk; end
else error()
until X=# /*栈空*/

```



语法分析的流程算法

三、设计目的:

- (1) 理解和掌握 LL(1)语法分析方法的基本原理; 根据给出的 LL(1)文法, 掌握 LL(1)分析表的构造及分析过程的实现。
- (2) 掌握预测分析程序如何使用分析表和栈联合控制实现 LL(1)分析。

四、实现环境和要求

选择实习环境为 486 以上 CPU,4M 内存,TURBO C2.0 语言. 实现程序见附录.

具体的实现要求:

- (1) 对输入文法, 它能判断是否为 LL(1)文法, 若是, 则转 (2); 否则报错并终止;
- (2) 输入已知文法, 由程序自动生成它的 LL(1)分析表;
- (3) 对于给定的输入串, 应能判断识别该串是否为给定文法的句型。

五、总结

上机前应做好准备. 即根据实习目的、要求和分析, 选择相应的数据结构, 使用 C 语言参照算法中的流程编写词法分析的程序. 将编好的程序上机进行调试. 注意调试的例子应有词法正确的, 也应有词法错误的或是超出所选数据结构范围的.

实验完成达到实习目的之后, 若尚有余力者, 可以对所选子集适当扩大或是增加相应功能如: ①扩充界符和保留字数目; ②允许实型常数; ③进行词法错误检查; ④最大范围扩充以至 PASCAL 语言所有字符的集合.

通过这次程序设计, 更加清楚透彻的明白了 L L (1)分析法的过程, 从而也比较熟练掌握了自上而下语法分析的基本思想, 此外, 巩固了所学的数据结构的知识, 自己所学的知识能够联系起来, 使得知识自成系统. 在实现和调试时采取模块化的思想, 使得本次课程设计比较顺利, 增强了自己的信心, 提高了自己的编程能力和动手能力以及独立分析问题、解决问题的能力 and 综合运用所学知识的能力。

附录

/******

预测分析程序 (语法分析程序), 分析对象为 C 语言源程序文件。

该分析程序有 18 部分组成:

- 《1》 首先定义各种需要用到的常量和变量;
- 《2》 判断一个字符是否在指定字符串中;
- 《3》 得到一个不是非终结符的符号;
- 《4》 分解含有左递归的产生式;
- 《5》 分解不含有左递归的产生式;
- 《6》 读入一个文法;
- 《7》 将单个符号或符号串并入另一符号串;
- 《8》 求所有能直接推出 $\wedge$ 的符号;
- 《9》 求某一符号能否推出 $\wedge$;
- 《10》 判断读入的文法是否正确;
- 《11》 求单个符号的 FIRST;
- 《12》 求各产生式右部的 FIRST;

- 《13》 求各产生式左部的 FOLLOW;
- 《14》 判断读入文法是否为一个 LL(1)文法;
- 《15》 构造分析表 M;
- 《16》 总控算法;
- 《17》 一个用户调用函数;
- 《18》 主函数;

```
#include<stdlib.h>
#include<stdio.h>
#include<string.h>

int count=0;           /*分解的产生式的个数*/
int number;            /*所有终结符和非终结符的总数*/
char start;            /*开始符号*/
char termin[50];       /*终结符号*/
char non_ter[50];      /*非终结符号*/
char v[50];            /*所有符号*/
char left[50];         /*左部*/
char right[50][50];    /*右部*/
char first[50][50],follow[50][50]; /*各产生式右部的 FIRST 和左部的 FOLLOW 集合*/
char first1[50][50];   /*所有单个符号的 FIRST 集合*/
char select[50][50];   /*各单个产生式的 SELECT 集合*/
char f[50],F[50];      /*记录各符号的 FIRST 和 FOLLOW 是否已求过*/
char empty[20];        /*记录可直接推出^的符号*/
char TEMP[50];         /*求 FOLLOW 时存放某一符号串的 FIRST 集合*/
int validity=1;        /*表示输入文法是否有效*/
int ll=1;              /*表示输入文法是否为 LL(1)文法*/
int M[20][20];         /*分析表*/
char choose;           /*用户输入时使用*/
char emp[20];          /*求 _emp()时使用*/
char fo[20];           /*求 FOLLOW 集合时使用*/
```

```
/******
```

判断一个字符是否在指定字符串中

```
*****/
```

```
int in(char c,char *p)
{
    int i;
    if(strlen(p)==0)
        return(0);
    for(i=0;;i++)
    {
        if(p[i]==c)
            return(1);      /*若在, 返回 1*/
        if(i==strlen(p))
            return(0);      /*若不在, 返回 0*/
    }
}
```

```

    }
}

/*****

得到一个不是非终结符的符号

*****/

char c()
{
    char c='A';
    while(in(c,non_ter)==1)
        c++;
    return(c);
}

/*****

分解含有左递归的产生式

*****/

void recur(char *point)
{
    /*完整的产生式在 point[] 中*/
    int j,m=0,n=3,k;
    char temp[20],ch;
    ch=c();    /*得到一个非终结符*/
    k=strlen(non_ter);
    non_ter[k]=ch;
    non_ter[k+1]='\0';
    for(j=0;j<=strlen(point)-1;j++)
    {
        if(point[n]==point[0])
        {
            /*如果' '后的首符号和左部相同*/
            for(j=n+1;j<=strlen(point)-1;j++)
            {
                while(point[j]!='' && point[j]!='\0')
                    temp[m++]=point[j++];
                left[count]=ch;
                memcpy(right[count],temp,m);
                right[count][m]=ch;
                right[count][m+1]='\0';
                m=0;
                count++;
                if(point[j]=='')
                {
                    n=j+1;
                    break;
                }
            }
        }
    }
}

```

```

else
{
    /*如果'\'后的首符号和左部不同*/
    left[count]=ch;
    right[count][0]='^';
    right[count][1]='\0';
    count++;
    for(j=n;j<=strlen(point)-1;j++)
    {
        if(point[j]!='\')
            temp[m++]=point[j];
        else
        {
            left[count]=point[0];
            memcpy(right[count],temp,m);
            right[count][m]=ch;
            right[count][m+1]='\0';
            printf(" count=%d ",count);
            m=0;
            count++;
        }
    }
    left[count]=point[0];
    memcpy(right[count],temp,m);
    right[count][m]=ch;
    right[count][m+1]='\0';
    count++;
    m=0;
}
}
}

```

/*****

分解不含有左递归的产生式

*****/

void non_re(char *point)

```

{
    int m=0,j;
    char temp[20];
    for(j=3;j<=strlen(point)-1;j++)
    {
        if(point[j]!='\')
            temp[m++]=point[j];
        else
        {
            left[count]=point[0];

```

```

        memcpy(right[count],temp,m);
        right[count][m]='\0';
        m=0;
        count++;
    }
}
left[count]=point[0];
memcpy(right[count],temp,m);
right[count][m]='\0';
count++;
m=0;
}
/*****

读入一个文法
*****/

char grammer(char *t,char *n,char *left,char right[50][50])
{
    char vn[50],vt[50];
    char s;
    char p[50][50];
    int i,j,k;
    printf("\n 请输入文法的非终结符号串： ");
    scanf("%s",vn);
    getchar();
    i=strlen(vn);
    memcpy(n,vn,i);
    n[i]='\0';
    printf("请输入文法的终结符号串： ");
    scanf("%s",vt);
    getchar();
    i=strlen(vt);
    memcpy(t,vt,i);
    t[i]='\0';
    printf("请输入文法的开始符号： ");
    scanf("%c",&s);
    getchar();
    printf("请输入文法产生式的条数： ");
    scanf("%d",&i);
    getchar();
    for(j=1;j<=i;j++)
    {
        printf("请输入文法的第%d 条 （共%d 条） 产生式： ",j,i);
        scanf("%s",p[j-1]);
        getchar();
    }
}

```



```

    }
    for(j=0;j<=i-1;j++)
        if(p[j][1]!='\0' || p[j][2]!='>')
        {
            printf("\ninput error!");
            validity=0;
            return('\0');
        }
        /*检测输入错误*/
    for(k=0;k<=i-1;k++)
    {
        /*分解输入的各产生式*/
        if(p[k][3]==p[k][0])
            recur(p[k]);
        else
            non_re(p[k]);
    }
    return(s);
}

/*****

将单个符号或符号串并入另一符号串
*****/

void merge(char *d,char *s,int type)
{
    /*d 是目标符号串，s 是源串，type=1，源串中的‘ ^ ’一并并入目串；
    type=2，源串中的‘ ^ ’不并入目串*/

    int i,j;
    for(i=0;i<=strlen(s)-1;i++)
    {
        if(type==2&& s[i]!='^')
            ;
        else
        {
            for(j=0;;j++)
            {
                if(j<strlen(d)&& s[i]==d[j])
                    break;
                if(j==strlen(d))
                {
                    d[j]=s[i];
                    d[j+1]='\0';
                    break;
                }
            }
        }
    }
}

/*****

```

求所有能直接推出 $\wedge$ 的符号

*****/

void emp(char c)

{ /*即求所有由 $\wedge$ 推出的符号*/

char temp[10];

int i;

for(i=0;i<=count-1;i++)

{

if(right[i][0]==c&&strlen(right[i])==1)

{

temp[0]=left[i];

temp[1]='\0';

merge(empty,temp,1);

emp(left[i]);

}

}

}

*****/

求某一符号能否推出 $\wedge$

*****/

int _emp(char c)

{ /*若能推出，返回 1；否则，返回 0*/

int i,j,k,result=1,mark=0;

char temp[20];

temp[0]=c;

temp[1]='\0';

merge(empty,temp,1);

if(in(c,empty)==1)

return(1);

for(i=0;;i++)

{

if(i==count)

return(0);

if(left[i]==c) /*找一个左部为 c 的产生式*/

{

j=strlen(right[i]); /*j 为右部的长度*/

if(j==1&&in(right[i][0],empty)==1)

return(1);

else if(j==1&&in(right[i][0],termin)==1)

return(0);

else

{

for(k=0;k<=j-1;k++)

if(in(right[i][k],empty)==1)

```

        mark=1;
    if(mark==1)
        continue;
    else
    {
        for(k=0;k<=j-1;k++)
        {
            result*=_emp(right[i][k]);
            temp[0]=right[i][k];
            temp[1]='\0';
            merge(empty,temp,1);
        }
    }
    if(result==0&& i<count)
        continue;
    else if(result==1&& i<count)
        return(1);
}

}

}

/*****
判断读入的文法是否正确
*****/

int judge()
{
    int i,j;
    for(i=0;i<=count-1;i++)
    {
        if(in(left[i],non_ter)==0)
        {
            /*若左部不在非终结符中，报错*/
            printf("\nerror1!");
            validity=0;
            return(0);
        }
        for(j=0;j<=strlen(right[i])-1;j++)
        {
            if(in(right[i][j],non_ter)==0&& in(right[i][j],termin)==0&& right[i][j]!='^')
            {
                /*若右部某一符号不在非终结符、终结符中且不为‘^’，报错*/
                printf("\nerror2!");
                validity=0;
                return(0);
            }
        }
    }
}

```

```

    }
    return(1);
}
/*****

求单个符号的 FIRST
*****/

void first2(int i)
{
    /*i 为符号在所有输入符号中的序号*/
    char c,temp[20];
    int j,k,m;
    c=v[i];
    char ch='^';
    emp(ch);
    if(in(c,termin)==1)      /*若为终结符*/
    {
        first1[i][0]=c;
        first1[i][1]='\0';
    }
    else if(in(c,non_ter)==1) /*若为非终结符*/
    {
        for(j=0;j<=count-1;j++)
        {
            if(left[j]==c)
            {
                if(in(right[j][0],termin)==1||right[j][0]=='^')
                {
                    temp[0]=right[j][0];
                    temp[1]='\0';
                    merge(first1[i],temp,1);
                }
                else if(in(right[j][0],non_ter)==1)
                {
                    if(right[j][0]==c)
                        continue;
                    for(k=0;;k++)
                        if(v[k]==right[j][0])
                            break;
                    if(f[k]=='0')
                    {
                        first2(k);
                        f[k]='1';
                    }
                    merge(first1[i],first1[k],2);
                    for(k=0;k<=strlen(right[j])-1;k++)

```

```

        {
            empt[0]='\0';
            if(!_emp(right[j][k])==1&& k<strlen(right[j])-1)
            {
                for(m=0;;m++)
                    if(v[m]==right[j][k+1])
                        break;
                if(f[m]=='0')
                {
                    first2(m);
                    f[m]='1';
                }
                merge(first1[i],first1[m],2);
            }
            else if(!_emp(right[j][k])==1&& k==strlen(right[j])-1)
            {
                temp[0]='^';
                temp[1]='\0';
                merge(first1[i],temp,1);
            }
            else
                break;
        }
    }
}

f[i]='1';
}

```

/******

求各产生式右部的 FIRST

*****/

void FIRST(int i,char *p)

```

{
    int length;
    int j,k,m;
    char temp[20];
    length=strlen(p);
    if(length==1)                /*如果右部为单个符号*/
    {
        if(p[0]=='^')
        {
            if(i>=0)
            {

```

```

        first[i][0]='^';
        first[i][1]='\0';
    }
    else
    {
        TEMP[0]='^';
        TEMP[1]='\0';
    }
}
else
{
    for(j=0;;j++)
        if(v[j]==p[0])
            break;
    if(i>=0)
    {
        memcpy(first[i],first1[j],strlen(first1[j]));
        first[i][strlen(first1[j])]='\0';
    }
    else
    {
        memcpy(TEMP,first1[j],strlen(first1[j]));
        TEMP[strlen(first1[j])]='\0';
    }
}
}
else /*如果右部为符号串*/
{
    for(j=0;;j++)
        if(v[j]==p[0])
            break;
    if(i>=0)
        merge(first[i],first1[j],2);
    else
        merge(TEMP,first1[j],2);
    for(k=0;k<=length-1;k++)
    {
        empt[0]='\0';
        if(!_emp(p[k])==1&& k<length-1)
        {
            for(m=0;;m++)
                if(v[m]==right[i][k+1])
                    break;
            if(i>=0)

```

```

        merge(first[i],first1[m],2);
    else
        merge(TEMP,first1[m],2);
    }
    else if(_emp(p[k])==1&& k==length-1)
    {

        temp[0]='^';
        temp[1]='\0';
        if(i>=0)
            merge(first[i],temp,1);
        else
            merge(TEMP,temp,1);
    }
    else if(_emp(p[k])==0)
        break;
    }
}
}
}

/*****

求各产生式左部的 FOLLOW
*****/

void FOLLOW(int i)
{
    int j,k,m,n,result=1;
    char c,temp[20];
    c=non_ter[i];          /*c 为待求的非终结符*/
    temp[0]=c;
    temp[1]='\0';
    merge(fo,temp,1);
    if(c==start)
    {
        /*若为开始符号*/
        temp[0]='#';
        temp[1]='\0';
        merge(follow[i],temp,1);
    }
    for(j=0;j<=count-1;j++)
    {
        if(in(c,right[j])==1)    /*找一个右部含有 c 的产生式*/
        {
            for(k=0;;k++)
                if(right[j][k]==c)
                    break;        /*k 为 c 在该产生式右部的序号*/
            for(m=0;;m++)

```

```

        if(v[m]==left[j])
            break;          /*m 为产生式左部非终结符在所有符号中的序号*/
    if(k==strlen(right[j])-1)
    {
        /*如果 c 在产生式右部的最后*/
        if(in(v[m],fo)==1)
        {
            merge(follow[i],follow[m],1);
            continue;
        }
        if(F[m]=='0')
        {
            FOLLOW(m);
            F[m]='1';
        }
        merge(follow[i],follow[m],1);
    }
    else
    {
        /*如果 c 不在产生式右部的最后*/
        for(n=k+1;n<=strlen(right[j])-1;n++)
        {
            empt[0]='\0';
            result*=_emp(right[j][n]);
        }
        if(result==1)
        {
            /*如果右部 c 后面的符号串能推出^*/
            if(in(v[m],fo)==1)
            {
                /*避免循环递归*/
                merge(follow[i],follow[m],1);
                continue;
            }
            if(F[m]=='0')
            {
                FOLLOW(m);
                F[m]='1';
            }
            merge(follow[i],follow[m],1);
        }
        for(n=k+1;n<=strlen(right[j])-1;n++)
            temp[n-k-1]=right[j][n];
        temp[strlen(right[j])-k-1]='\0';
        FIRST(-1,temp);
        merge(follow[i],TEMP,2);
    }
}
}

```



```

    }
    F[i]='1';
}

/*****
判断读入文法是否为一个 LL(1)文法
*****/

int ll1()
{
    int i,j,length,result=1;
    char temp[50];
    for(j=0;j<=49;j++)
    {
        /*初始化*/
        first[j][0]='\0';
        follow[j][0]='\0';
        first1[j][0]='\0';
        select[j][0]='\0';
        TEMP[j]='\0';
        temp[j]='\0';
        f[j]='0';
        F[j]='0';
    }
    for(j=0;j<=strlen(v)-1;j++)
        first2(j);          /*求单个符号的 FIRST 集合*/
    printf("\nfirst1:");
    for(j=0;j<=strlen(v)-1;j++)
        printf("%c:%s  ",v[j],first1[j]);
    printf("\nempty:%s",empty);
    printf("\n::\n_emp:");
    for(j=0;j<=strlen(v)-1;j++)
        printf("%d  ",_emp(v[j]));
    for(i=0;i<=count-1;i++)
        FIRST(i,right[i]);  /*求 FIRST*/
    printf("\n");
    for(j=0;j<=strlen(non_ter)-1;j++)
    {
        /*求 FOLLOW*/
        if(fo[j]==0)
        {
            fo[0]='\0';
            FOLLOW(j);
        }
    }
    printf("\nfirst:");
    for(i=0;i<=count-1;i++)

```

```

        printf("%s ",first[i]);
    printf("\nfollow:");
    for(i=0;i<=strlen(non_ter)-1;i++)
        printf("%s ",follow[i]);
    for(i=0;i<=count-1;i++)
    {
        /*求每一产生式的 SELECT 集合*/
        memcpy(select[i],first[i],strlen(first[i]));
        select[i][strlen(first[i])]='\0';
        for(j=0;j<=strlen(right[i])-1;j++)
            result*=_emp(right[i][j]);
        if(strlen(right[i])==1&&right[i][0]=='^')
            result=1;
        if(result==1)
        {
            for(j=0;j++)
                if(v[j]==left[i])
                    break;
            merge(select[i],follow[j],1);
        }
    }
    printf("\nselect:");
    for(i=0;i<=count-1;i++)
        printf("%s ",select[i]);
    memcpy(temp,select[0],strlen(select[0]));
    temp[strlen(select[0])]='\0';
    for(i=1;i<=count-1;i++)
    {
        /*判断输入文法是否为 LL(1)文法*/
        length=strlen(temp);
        if(left[i]==left[i-1])
        {
            merge(temp,select[i],1);
            if(strlen(temp)<length+strlen(select[i]))
                return(0);
        }
        else
        {
            temp[0]='\0';
            memcpy(temp,select[i],strlen(select[i]));
            temp[strlen(select[i])]='\0';
        }
    }
    return(1);
}

```

```
/******
```

构造分析表 M

```
*****/
```

```
void MM()
```

```
{
    int i,j,k,m;
    for(i=0;i<=19;i++)
        for(j=0;j<=19;j++)
            M[i][j]=-1;
    i=strlen(termin);
    termin[i]='#';    /*将#加入终结符数组*/
    termin[i+1]='\0';
    for(i=0;i<=count-1;i++)
    {
        for(m=0;;m++)
            if(non_ter[m]==left[i])
                break;    /*m 为产生式左部非终结符的序号*/
        for(j=0;j<=strlen(select[i])-1;j++)
        {
            if(in(select[i][j],termin)==1)
            {
                for(k=0;;k++)
                    if(termin[k]==select[i][j])
                        break;    /*k 为产生式右部终结符的序号*/
                M[m][k]=i;
            }
        }
    }
}
```

```
/******
```

总控算法

```
*****/
```

```
void syntax()
```

```
{
    int i,j,k,m,n,p,q;
    char ch;
    char S[50],str[50];
    printf("请输入该文法的句型: ");
    scanf("%s",str);
    getchar();
    i=strlen(str);
    str[i]='#';
    str[i+1]='\0';
```

```

S[0]='#';
S[1]=start;
S[2]='\0';
j=0;
ch=str[j];
while(1)
{
    if(in(S[strlen(S)-1],termin)==1)
    {
        if(S[strlen(S)-1]!=ch)
        {
            printf("\n 该符号串不是文法的句型！");
            return;
        }
        else if(S[strlen(S)-1]=='#')
        {
            printf("\n 该符号串是文法的句型.");
            return;
        }
        else
        {
            S[strlen(S)-1]='\0';
            j++;
            ch=str[j];
        }
    }
    else
    {
        for(i=0;;i++)
            if(non_ter[i]==S[strlen(S)-1])
                break;
        for(k=0;;k++)
        {
            if(termin[k]==ch)
                break;
            if(k==strlen(termin))
            {
                printf("\n 词法错误！");
                return;
            }
        }
        if(M[i][k]==-1)
        {
            printf("\n 语法错误！");

```

```

        return;
    }
    else
    {
        m=M[i][k];
        if(right[m][0]=='^')
            S[strlen(S)-1]='\0';
        else
        {
            p=strlen(S)-1;
            q=p;
            for(n=strlen(right[m])-1;n>=0;n--)
                S[p++]=right[m][n];
            S[q+strlen(right[m])]='\0';
        }
    }
}

printf("\nS:%s str:",S);
for(p=j;p<=strlen(str)-1;p++)
    printf("%c",str[p]);
printf(" ");
}
}

```

/******

一个用户调用函数

*****/

void menu()

```

{
    syntax();
    printf("\n 是否继续? (y or n):");
    scanf("%c",&choose);
    getchar();
    while(choose=='y')
    {
        menu();
    }
}

```

/******

主函数

*****/

void main()

```

{
    int i,j;
    start=grammar(termin,non_ter,left,right);          /*读入一个文法*/
    printf("count=%d",count);
    printf("\nstart:%c",start);
    strcpy(v,non_ter);
    strcat(v,termin);
    printf("\nv:%s",v);
    printf("\nnon_ter:%s",non_ter);
    printf("\ntermin:%s",termin);
    printf("\nright:");
    for(i=0;i<=count-1;i++)
        printf("%s  ",right[i]);
    printf("\nleft:");
    for(i=0;i<=count-1;i++)
        printf("%c  ",left[i]);
    if(validity==1)
        validity=judge();
    printf("\nvalidity=%d",validity);
    if(validity==1)
    {
        printf("\n 文法有效");
        ll=ll1();
        printf("\nll=%d",ll);
        if(ll==0)
            printf("\n 该文法不是一个 LL1 文法 ! ");
        else
        {
            MM();
            printf("\n");
            for(i=0;i<=19;i++)
                for(j=0;j<=19;j++)
                    if(M[i][j]>=0)
                        printf("M[%d][%d]=%d ",i,j,M[i][j]);
            printf("\n");
            menu();
        }
    }
}

```

5.执行结果

(1) 输入一个文法

```

D:\Turboc2\Debug\syntax.exe 陶晶莹 - 太委屈
请输入文法的非终结符号串: ETF
请输入文法的终结符号串: +-*/<>i
请输入文法的开始符号: E
请输入文法产生式的条数: 3
请输入文法的第1条 (共3条) 产生式: E->E+T!E-T!T
请输入文法的第2条 (共3条) 产生式: T->T*F!T/F!F
请输入文法的第3条 (共3条) 产生式: F-><E>!i
count=10
start:E
v:ETFAB+-*/<>i
non_ter:ETFAB
termin:+-*/<>i
right:+TA -TA ^ TA *FB /FB ^ FB <E> i
left:A A A E B B B T F F
validity=1
first1:E:<i T:<i F:<i A:+-^ B:*/^ +:+ -:- *:*/ /:/ <:< >:> i:i
empty:AB
_empt:0 0 0 1 1 0 0 0 0 0 0 0
first:+ - ^ <i * / ^ <i < i
follow:#> >#+- +>#-*/ ># +>#-
select:+ - ^># <i * / ^>#- <i < i
ll=1
M[0][4]=3 M[0][6]=3 M[1][4]=7 M[1][6]=7 M[2][4]=8 M[2][6]=9 M[3][0]=0 M[3][1]=1
M[3][5]=2 M[3][7]=2 M[4][0]=6 M[4][1]=6 M[4][2]=4 M[4][3]=5 M[4][5]=6 M[4][7]=6
请输入该文法的句型:

```

(2) 输入一个符号串

```

D:\Turboc2\Debug\syntax.exe 陶晶莹 - 太委屈
请输入该文法的句型: i*(i+i)
S:#AT str:i*(i+i)#
S:#ABF str:i*(i+i)#
S:#ABi str:i*(i+i)#
S:#AB str:*(i+i)#
S:#ABF* str:*(i+i)#
S:#ABF str:(i+i)#
S:#AB)E< str:(i+i)#
S:#AB)E str:i+i)#
S:#AB)AT str:i+i)#
S:#AB)ABF str:i+i)#
S:#AB)ABi str:i+i)#
S:#AB)AB str:+i)#
S:#AB)A str:+i)#
S:#AB)AT+ str:+i)#
S:#AB)AT str:i)#
S:#AB)ABF str:i)#
S:#AB)ABi str:i)#
S:#AB)AB str:>#
S:#AB)A str:>#
S:#AB) str:>#
S:#AB str:#
S:#A str:#
S:# str:#
该符号串是文法的句型.
是否继续? (y or n):

```

(3) 再次输入一个符号串，然后退出程序

```
C:\ "D:\Turbo2\Debug\syntax.exe" F4 - 第一时间
是否继续? (y or n):y
请输入该文法的句型: i+(i*i)i
S:#AT str:i+(i*i)i#
S:#ABF str:i+(i*i)i#
S:#ABi str:i+(i*i)i#
S:#AB str:+(i*i)i#
S:#A str:+(i*i)i#
S:#AT+ str:+(i*i)i#
S:#AT str:(i*i)i#
S:#ABF str:(i*i)i#
S:#AB>E< str:(i*i)i#
S:#AB>E str:i*i)i#
S:#AB>AT str:i*i)i#
S:#AB>ABF str:i*i)i#
S:#AB>ABi str:i*i)i#
S:#AB>AB str:*i)i#
S:#AB>ABF* str:*i)i#
S:#AB>ABF str:i)i#
S:#AB>ABi str:i)i#
S:#AB>AB str:)>i#
S:#AB>A str:)>i#
S:#AB> str:)>i#
S:#AB str:i#
语法错误!
是否继续? (y or n):n
Press any key to continue_
```